Universidade Estadual da Paraíba

Centro de Ciências e Tecnologia

Departamento de Computação

Disciplina: Técnicas de Análise de Algoritmos

Professor: Fábio Luiz Leite Júnior

Emanuel Diogo Marinho de Souza

Filipe Antonny Tavares de Santana

Rafael Trajano do Nascimento

Projeto de Técnicas de Análise de Algoritmos

Problema: Alocação de Salas de Aula - Estratégia Gulosa e Programação Dinâmica

Campina Grande – PB

05 de março de 2026

Sumário

1. Introdução……3
2. Metodologia e implementação……4
3. Algoritmos implementados……...5
   3.1 Estratégia Gulosa...……5
      3.1.1 Análise de corretude……6
      3.1.2 Análise de desempenho…….…6
   3.2 Programação Dinâmica……6
      3.2.1 Análise de corretude……...7
      3.2.2 Análise de desempenho……7
4. Experimentos e resultados……8
5. Análise e discussão……17
6. Conclusão……20
7. Referências……...21

1. Introdução

O problema de alocação de salas de aula consiste em determinar o número mínimo de salas necessárias para acomodar um conjunto de aulas, considerando seus horários de início e término. Duas aulas que possuem horários sobrepostos não podem ocorrer na mesma sala. Esse tipo de problema aparece em diversas áreas, como planejamento acadêmico, organização de eventos e escalonamento de tarefas.

Neste relatório, são analisadas e comparadas quatro técnicas fundamentais de projeto de algoritmos: **Backtracking** e **Branch and Bound** (abordadas na primeira etapa), e **Programação Dinâmica** e **Estratégia Gulosa** (foco desta segunda etapa). Todas as técnicas foram implementadas na linguagem Java. O objetivo final é realizar um estudo comparativo abrangente, avaliando o comportamento de cada paradigma em termos de corretude, eficiência computacional e capacidade de lidar com instâncias de diferentes tamanhos.

2. Metodologia e Implementação

O sistema foi desenvolvido utilizando a linguagem de programação Java. O programa recebe como entrada o número de aulas e os horários de início e término de cada uma delas. A partir dessas informações, os algoritmos implementados determinam o número mínimo de salas necessárias e, quando aplicável, realizam a alocação das aulas em salas disponíveis.

A entrada do programa consiste em um número inteiro n, representando a quantidade de aulas, seguido de n linhas, cada uma contendo dois valores inteiros correspondentes ao horário de início e ao horário de término de cada aula.

A saída do programa apresenta o número mínimo de salas necessárias e, nos casos em que há construção explícita da solução, a atribuição de cada aula a uma sala específica.

3. Algoritmos Implementados

Nesta segunda etapa do trabalho, o foco está na análise de algoritmos mais eficientes para o problema de alocação de aulas em salas, com ênfase em abordagens de complexidade polinomial.

Os algoritmos de **Backtracking** e **Branch and Bound** foram implementados e analisados na primeira parte do projeto, servindo como base para comparação com as abordagens atuais.

Dessa forma, nesta etapa são exploradas as seguintes estratégias:

- Estratégia Gulosa
- Programação Dinâmica

3.1 Estratégia Gulosa

A estratégia gulosa resolve o problema por meio de escolhas locais ótimas, buscando construir uma solução global eficiente. Inicialmente, as aulas são ordenadas pelo horário de início. Em seguida, cada aula é alocada na primeira sala disponível cujo horário de término seja compatível com o início da aula atual.

```
if (!pq.isEmpty() && pq.peek()[0] <= aula.getInicio()) {

   int[] sala = pq.poll();
   alocacao[aula.getId()] = sala[1];
   pq.add(new int[]{aula.getFim(), sala[1]});

}
```

A estrutura de dados fundamental utilizada é uma **fila de prioridade (min-heap)**, que permite acessar eficientemente a sala com menor tempo de término:

```
PriorityQueue<int[]> pq = new PriorityQueue
<>(Comparator.comparingInt(a->a[0]));
```

Nesse contexto, o algoritmo sempre reutiliza a sala que se torna disponível primeiro, evitando a criação desnecessária de novas salas.

```
else {
    salas++;
    alocacao[aula.getId()] = salas;

    pq.add(new int[]{aula.getFim(), salas});
}
```

Quando não há sala disponível, uma nova sala é criada.

3.1.1 Análise de Corretude

A corretude do algoritmo baseia-se na ordenação das aulas por horário de início e na escolha gulosa de sempre reutilizar a sala que libera mais cedo.

Essa estratégia garante que:

- Não ocorram conflitos de horário;
- O número de salas utilizadas seja mínimo.

A fila de prioridade mantém corretamente o estado das salas em uso, assegurando que a decisão tomada em cada passo seja válida.

3.1.2 Análise de Desempenho

Complexidade Espacial: O(n)

Complexidade Temporal: O(n log n)

3.2 Programação Dinâmica

Essa abordagem utiliza programação dinâmica para calcular o número mínimo de salas necessárias para alocar todas as aulas sem conflito. Inicialmente, os horários de início e término das aulas são separados e ordenados. Em seguida, é realizada uma varredura linear:

```
int i = 0, j = 0;

int salasAtivas = 0;

int maxSalas = 0;
```

```
while (i < n && j < n) {
    if (inicios[i] < fins[j]) {
        salasAtivas++;
        maxSalas = Math.max(maxSalas, salasAtivas);
        i++;
    } else {
        salasAtivas--;
        j++;
    }
}
```

Quando uma aula começa antes de outra terminar, aumenta-se o número de salas em uso. Caso contrário, uma sala é liberada.
Essa abordagem calcula apenas o número mínimo de salas necessárias, sem construir explicitamente a alocação das aulas.

3.2.1 Análise de Corretude

A corretude do algoritmo baseia-se na interpretação do problema como a determinação do número máximo de intervalos sobrepostos em um dado instante.
Ao ordenar separadamente os horários de início e término e percorrê-los de forma sincronizada, o algoritmo contabiliza corretamente o número de aulas simultâneas a cada instante.
A variável `salasAtivas` mantém o número de aulas em andamento, enquanto `maxSalas` armazena o maior valor observado durante a execução. Esse valor corresponde exatamente ao número mínimo de salas necessárias para evitar conflitos.

3.2.2 Análise de Desempenho

- **Complexidade Temporal:** O(n log n), devido à ordenação dos vetores de horários de início e término. A etapa de varredura é linear (O(n)).
- **Complexidade Espacial:** O(n), necessária para armazenar os vetores de horários de início e término.

4. Experimentos e Resultados

Para avaliar o funcionamento dos algoritmos implementados, foi utilizado um conjunto de aulas com horários de início e término definidos. O objetivo foi verificar quantas salas seriam necessárias para acomodar todas as aulas sem que ocorressem conflitos de horário.

É importante ressaltar que as entradas usadas para 6 aulas, 10 aulas e 20 aulas foram as mesmas usadas para Backtracking e Branch and Bound.

O número inteiro da primeira linha é a quantidade de aulas a serem alocadas nas salas, as linhas seguintes representam cada aula, contendo o horário de início e o horário de término.

O resultado da alocação das salas para a quantidade de aulas está seguindo os algoritmos de Programação Dinâmica e Estratégia gulosa, em verde temos a entrada para a execução dos algoritmos e logo após a saída dos mesmos.

```
6
9 10
9 12
11 13
12 14
13 15
14 16
```

```
Greedy:
2
1 1
2 2
3 1
4 2
5 1
6 2
Tempo: 0,014277 segundos
```

```
Programação Dinâmica:
Número mínimo de salas usadas: 2
Tempo: 0,006360 segundos
```

Imagem 1: Teste feito com 6 aulas

```
10
7 8
7 9
9 10
9 11
9 14
7 8
10 12
10 11
7 8
8 9
```

```
Greedy:
4
1 1
2 2
3 4
4 3
5 2
6 3
7 1
8 4
9 4
10 1
Tempo: 0,023281 segundos
```

```
Programação Dinâmica:
Número mínimo de salas usadas: 4
Tempo: 0,006391 segundos
```

Imagem 2: Teste feito com 10 aulas

```
20
7 8
7 9
9 12
8 9
10 12
13 14
16 18
10 12
7 9
8 10
7 9
10 12
20 22
7 9
10 11
10 12
12 13
13 15
7 11
11 13
```

```
Greedy:
7
1 1
2 2
3 2
4 1
5 4
6 5
7 2
8 5
9 3
10 7
11 4
12 3
13 7
14 5
15 1
16 7
17 1
18 3
19 6
20 6
Tempo: 0,025697 segundos
Programação Dinâmica:
Número mínimo de salas usadas: 7
Tempo: 0,008915 segundos
```

Imagem 3: Teste feito com 20 aulas

```
30
7 8
7 9
10 12
18 19
20 22
7 9
9 10
11 12
13 14
15 16
20 22
8 9
9 11
12 14
11 13
16 18
8 10
10 11
9 10
9 11
18 20
15 16
15 18
20 22
7 9
8 10
10 11
12 15
16 18
7 8
```

```
Greedy:
6
1 1
2 2
3 1
4 3
5 4
6 3
7 1
8 4
9 6
10 1
11 6
12 1
13 2
14 2
15 5
16 6
17 5
18 6
19 4
20 3
21 1
22 4
23 5
24 2
25 4
26 6
27 5
28 3
29 2
30 5
Tempo: 0,016939 segundos
```

```
Programação Dinâmica:
Número mínimo de salas usadas: 6
Tempo: 0,009576 segundos
```

Imagem 4: Teste feito com 30 aulas

```
50
7 8
7 9
7 10
8 9
10 11
18 19
15 16
10 12
9 12
18 19
20 22
21 22
18 20
7 9
10 11
10 12
8 10
9 10
7 10
15 16
13 15
13 15
20 22
19 21
18 22
7 12
12 14
13 18
12 13
7 10
7 9
10 11
10 12
10 13
7 9
7 8
7 10
10 12
13 15
18 22
18 19
18 19
18 20
18 20
18 20
20 22
20 22
20 21
20 21
7 9
```

```
Greedy:
12
1 1
2 2
3 3
4 1
5 9
6 10
7 1
8 2
9 4
10 5
11 9
12 6
13 2
14 4
15 12
16 1
17 10
18 8
19 5
20 4
21 12
22 8
23 11
24 4
25 3
26 6
27 11
28 9
29 7
30 7
31 8
32 8
33 10
34 3
35 9
36 10
37 11
38 5
39 6
40 7
41 11
42 8
43 6
44 12
45 1
46 5
47 10
48 8
49 1
50 12
Tempo: 0,025403 segundos
```

```
Programação Dinâmica:
Número mínimo de salas usadas: 12
Tempo: 0,009828 segundos
```

Imagem 5: Teste feito com 50 aulas

```
100
7 8
7 9
7 9
7 10
7 10
10 12
10 11
10 11
7 9
7 10
7 8
8 9
8 9
9 10
9 10
8 10
11 12
11 12
11 13
10 12
10 12
10 13
7 8
7 9
10 13
9 10
9 12
20 22
20 22
21 22
19 22
19 22
7 8
7 9
9 10
```

```
10 11
10 12
13 14
14 16
15 18
18 19
19 20
18 20
19 20
20 22
19 20
18 19
17 19
7 9
9 10
18 19
15 16
15 16
17 19
20 22
19 20
20 21
21 22
21 22
19 20
20 22
7 9
7 8
17 19
7 9
9 10
10 12
10 13
20 22
15 16
14 15
17 19
```

```
12 13
13 14
7 8
7 9
7 9
7 10
10 12
10 12
10 11
17 19
19 22
20 22
7 9
9 10
7 9
7 8
10 12
10 13
14 15
14 16
14 15
14 16
16 18
20 22
20 22
20 22
20 21
7 9
```

Imagem 6: Entradas para teste feito com 100 aulas

```
Greedy:
24
1 1
2 2
3 3
4 4
5 5
6 18
7 19
8 21
9 6
10 7
11 8
12 1
13 8
14 17
15 11
16 9
17 1
18 9
19 20
20 8
21 22
22 10
23 9
24 10
25 6
26 23
27 15
28 12
29 7
30 5
31 10
32 6
33 11
34 12
35 1
36 12
37 13
38 2
39 7
40 8
41 14
42 11
43 24
44 2
45 13
46 4
47 20
48 9
49 13
50 24
51 5
52 3
53 23
54 17
55 23
56 19
57 15
58 20
59 1
60 21
61 8
62 14
63 15
64 1
65 16
66 16
67 3
68 14
69 16
70 15
71 21
72 18
73 5
74 11
75 17
76 18
77 19
78 20
79 16
80 23
81 4
82 22
83 3
84 17
85 21
86 2
87 22
88 23
89 17
90 24
91 19
92 12
93 4
94 13
95 16
96 14
97 22
98 9
99 18
100 24
Tempo: 0,033311 segundos
```

Imagem 7: Saídas do Estratégia Gulosa para teste feito com 100 aulas

```
Programação Dinâmica:
Número mínimo de salas usadas: 24
Tempo: 0,010905 segundos
```

Imagem 8: Saída da Programação Dinâmica para teste feito com 100 aulas

Os testes realizados com diferentes tamanhos de entrada mostram claramente o comportamento dos dois algoritmos testados diante do crescimento do número de aulas. Observa-se que, de acordo com que a quantidade de aulas aumenta (6, 20, 30, 50 e 100 entradas), o algoritmo guloso (Greedy) mantém resultados corretos quanto ao número mínimo de salas, apresentando valores coerentes com a quantidade de conflitos simultâneos. Por exemplo, para entradas maiores como 50 e 100 aulas, os resultados encontrados (12 e 24 salas, respectivamente) mostram um crescimento proporcional ao crescimento da entrada do teste, confirmando a eficiência do método para esse tipo de problema.

Em relação ao desempenho, nota-se que o tempo de execução do algoritmo guloso cresce de forma moderada, mantendo-se sempre com tempo muito baixo (na casa de milissegundos). Isso ocorre porque sua complexidade é O(n log n), devido principalmente à ordenação inicial e ao uso da fila de prioridade. Mesmo com o aumento significativo da entrada, o tempo permanece estável e aceitável, demonstrando boa escalabilidade.

Por outro lado, a abordagem da Programação Dinâmica apresenta um comportamento mais eficiente em tempo de execução. Conforme observado nos resultados obtidos, seu tempo permanece praticamente constante e extremamente baixo em todos os testes, independentemente do tamanho da entrada. Isso se deve ao fato de que a solução implementada utiliza uma estratégia baseada em ordenação e varredura linear, resultando também em complexidade O(n log n), porém o mesmo não é a abordagem ideal para resolução do problema em questão, já que ele não aloca as aulas nas salas, apenas calcula e exibe a quantidade mínima de salas necessárias para a resolução do problema.

Além disso, é importante destacar que ambos os algoritmos (Greedy e Programação Dinâmica), assim como os anteriormente testados (Backtracking e Branch and Bound,até que fosse possível quanto a escalabilidade do problema) produziram exatamente o mesmo resultado para o número de salas em todas as instâncias testadas, o que valida a correção das implementações. No entanto, diferem conceitualmente: enquanto os algoritmos Guloso, Backtracking e Branch and Bound constroem explicitamente a alocação das aulas em salas, a programação dinâmica limita-se apenas a calcular a quantidade máxima de simultaneidade, sem fornecer a distribuição das aulas.

Desta forma, para o problema de alocação de salas, abordagens baseadas em ordenação e varredura são altamente eficientes e escaláveis, sendo mais indicadas do que métodos exaustivos (Backtracking e Branch and Bound - mesmo com otimizações). O algoritmo guloso se destaca por fornecer uma solução completa (alocação de aulas), enquanto a programação dinâmica apresenta melhor desempenho em tempo, sendo ideal quando apenas o número mínimo de salas é necessário.

5. Análise e Discussão

Os resultados obtidos demonstram que as quatro técnicas analisadas **Backtracking, Branch and Bound, Estratégia Gulosa e Programação Dinâmica** foram capazes de resolver corretamente o problema de alocação de salas de aula. Em todos os casos, os algoritmos convergiram para a solução ótima, identificando o número mínimo de salas necessário para acomodar o conjunto de aulas sem conflitos de horário.

Embora todos garantam a otimalidade para este problema específico, as técnicas divergem drasticamente na forma como exploram o espaço de soluções e no consumo de recursos computacionais:

Backtracking e Branch and Bound: Como discutido na primeira etapa, o Backtracking realiza uma busca exaustiva **(KLEINBERG; TARDOS, 2006)**, o que garante a análise de todas as possibilidades, mas compromete o tempo de execução em instâncias maiores. O Branch and Bound melhora esse cenário através de podas, mas ainda permanece vinculado a uma estrutura de árvore de decisão que pode crescer significativamente.

Estratégia Gulosa: Esta técnica destaca-se pela eficiência extrema. Ao ordenar as aulas por horário de início, o algoritmo toma decisões locais que, para este problema, resultam no ótimo global. Sua execução é quase instantânea, pois evita completamente a exploração de ramos desnecessários ou o preenchimento de tabelas complexas.

Programação Dinâmica: Oferece uma abordagem intermediária em termos de complexidade estrutural. Ao decompor o problema em subproblemas e utilizar a memorização, ela elimina a redundância do Backtracking, mantendo um tempo de execução polinomial e previsível.

A tabela a seguir apresenta uma comparação geral entre as técnicas utilizadas:

Critério	**Backtracking**	**Branch and Bound**	**Guloso**	**c**
Estratégia	Exploração completa (Exaustiva)	Exploração com poda de ramos	Decisão local ótima (Ordenação)	Decomposição e Memorização

Complexidade Teórica	$O(k^n)$ (Exponencial)	$O(k^n)$ (Pior caso)	O(n log n)	O(n log n)
Qualidade da Solução	Ótima	Ótima	Ótima	Ótima
Tempo de Execução	Muito Alto em instâncias grandes	Reduzido em relação ao Backtracking	Baixo (Milissegundos)	Mínimo (Aritmético)
Uso de Memória	O(n) (Pilha de recursão)	O(n)	O(n)	O(n)
Escalabilidade	Baixa	Média	Altíssima	Altíssima

Embora a literatura (CORMEN et al., 2009) aponte que a Programação Dinâmica para problemas de intervalos possa atingir complexidade $O(n^2)$, a implementação adotada neste trabalho utiliza uma **otimização de varredura de eventos**. Isso reduz o custo para O(n log n) e o uso de memória para $O(n)$, justificando por que esta abordagem apresentou o melhor desempenho empírico entre todos os algoritmos testados.

A análise comparativa evidencia que a escolha do paradigma de programação impacta diretamente a escalabilidade do sistema. Enquanto o **Backtracking** apresenta um crescimento exponencial de tempo que inviabiliza o uso em cenários reais com centenas de aulas, a **Estratégia Gulosa** demonstra ser a solução mais robusta para este problema, mantendo performance excepcional independentemente do volume de dados.

A **Programação Dinâmica**, embora exija um gerenciamento mais cuidadoso da memória para armazenar os subproblemas, mostra-se superior às técnicas de busca da primeira entrega, pois transforma um problema de complexidade

exponencial em um problema de complexidade polinomial. Em suma, para o problema de alocação de salas, as técnicas da segunda entrega (Gulosa e DP) superam as da primeira (Backtracking e B&B) em termos de viabilidade prática e eficiência empírica.

Esse comportamento também evidencia a escalabilidade dos algoritmos, uma vez que o Backtracking tende a apresentar crescimento exponencial com o aumento do número de aulas, enquanto o Branch and Bound reduz esse crescimento por meio de podas.

6. Conclusão

Neste projeto, foi analisado o problema de alocação de salas de aula, com o objetivo de determinar o número mínimo de salas necessárias para acomodar um conjunto de aulas sem conflitos de horários. Para resolver esse problema, foram implementadas e comparadas quatro técnicas fundamentais de projeto de algoritmos: **Backtracking, Branch and Bound, Estratégia Gulosa e Programação Dinâmica**, utilizando a linguagem de programação Java.

Os resultados obtidos demonstraram que todas as abordagens são capazes de encontrar a solução ótima para o problema proposto. No entanto, a análise revelou distinções profundas em termos de eficiência e viabilidade computacional. Enquanto o **Backtracking** e o **Branch and Bound** baseiam-se em explorações exaustivas de árvores de decisão o que resulta em baixa escalabilidade e tempos de execução elevados para grandes conjuntos de dados devido à sua natureza exponencial ou de alto custo as técnicas introduzidas, nesta segunda etapa apresentaram um desempenho superior.

A **Estratégia Gulosa** mostrou-se extremamente eficiente, alcançando a solução ótima com complexidade temporal de **O(n log n)**, o que é uma excelente escolha para problemas de larga escala. Paralelamente, a **Programação Dinâmica** ofereceu a abordagem mais veloz em termos empíricos, reduzindo o tempo de execução ao transformar um problema exponencial em um problema de tempo polinomial por meio de uma técnica otimizada de varredura.

Em termos de uso de memória, todas as técnicas mantiveram um consumo dentro de limites aceitáveis. Embora a Programação Dinâmica possa exigir maior armazenamento em implementações clássicas, a versão otimizada utilizada neste trabalho manteve a eficiência linear. A principal conclusão deste estudo é que a escolha do paradigma de programação, seja ele exaustivo, guloso ou baseado em subproblemas, impacta diretamente a escalabilidade da solução.

Dessa forma, este projeto evidencia que, para problemas de escalonamento como o de alocação de salas, a aplicação de estratégias gulosas ou de programação dinâmica é fundamental para garantir que sistemas reais operem com eficiência computacional, transformando desafios complexos em soluções ágeis e escaláveis.

7. Referências

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.

Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.
KLEINBERG, J.; TARDOS, É. Algorithm Design. Boston: Pears